

MODULE 4: STATIC & DYNAMIC MODELS

Contents

1. Models	1
2. UML	2
3. Static Models.....	3
4. Dynamic Models	4
5. State and Activity Models	5
6. Object-Oriented Modelling (Classes, Objects, and Messages)	6

Models

Modelling in software engineering refers to the process of creating abstract representations of a system. These representations, or "models," help software engineers understand, design, and document the system's structure, behaviour, and interactions. Models are used to simplify complex systems by focusing on key aspects while leaving out unnecessary details.

1. Purpose of Modelling:

- **Simplification:** Models reduce the complexity of real-world systems, making it easier to understand and manage them.
- **Communication:** Models provide a common language for developers, stakeholders, and analysts, aiding in clearer communication.
- **Documentation:** Models serve as documentation for the system's design, behaviour, and structure, which helps future developers and maintainers.
- **Analysis and Testing:** Models allow for the simulation of system behaviour, enabling analysis and testing of various scenarios before actual implementation.

2. Types of Models:

In software engineering, models are categorized based on their focus: **Static Models** and **Dynamic Models**.

i. Static Models:

- Focus on the structure of a system.
- Represent elements like objects, classes, components, and their relationships.
- Typically depict things that do not change over time, such as the system's architecture or data structure.
- Examples include class diagrams, entity-relationship diagrams, and component diagrams.

ii. Dynamic Models:

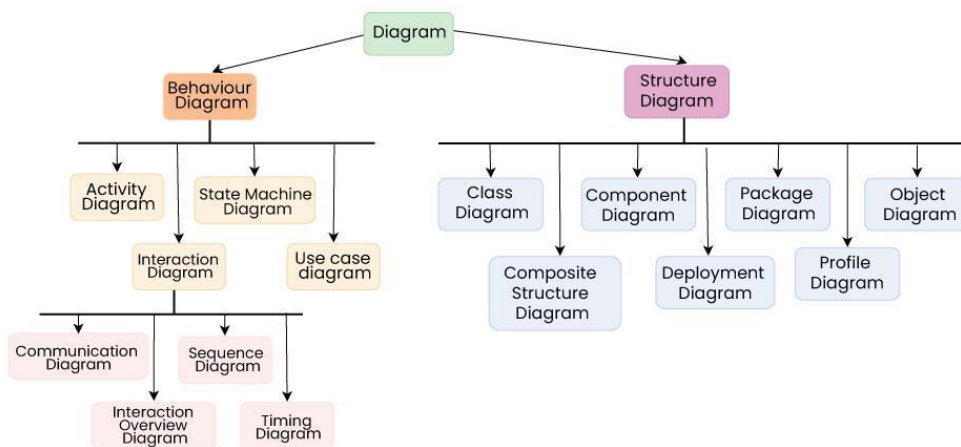
- Focus on the behaviour of a system over time.
- Represent how a system reacts to external and internal events, changes in state, and the interactions between system components.
- Examples include state diagrams, sequence diagrams, activity diagrams, and communication diagrams.

UML

In software engineering, UML (Unified Modelling Language) is a standardized visual language used to specify, visualize, construct, and document the components of software systems. It helps developers and stakeholders understand and communicate complex system designs by providing a structured way to create diagrams representing the system's structure and behaviour. Types of UML:

A. Behavioural Diagrams: Behavioural diagrams describe the dynamic behaviour of the system, capturing the interactions, activities, and state transitions.

1. **Activity Diagram:** Visualizes workflows and the sequence of activities within a process or operation.
2. **Interaction Diagrams**
 - **Communication Diagram:** Illustrates interactions between objects or components, showing their relationships and message flows.
 - **Interaction Overview Diagram:** Combines activity and interaction diagrams, showing a high-level control flow of interactions.
 - **Sequence Diagram:** Depicts the order of messages exchanged between objects over time in a specific scenario.
 - **Timing Diagram:** Shows the behaviour of objects over a time, focusing on timing constraints and object states.
3. **State Machine Diagram:** Captures the states of an object and transitions between states based on events, useful for systems with significant state-dependent behaviour.
4. **Use Case Diagram:** Models user interactions, representing the relationships between actors and use cases that define system functionalities.



UML Diagrams



B. Structural Diagrams: Structural diagrams focus on the static structure of the system, showing system components and their relationships.

1. **Class Diagram:** Defines the structure of classes, their attributes, methods, and relationships, providing a blueprint of the system's classes.
2. **Component Diagram:** Illustrates physical components (such as modules and libraries) and their dependencies, commonly used for implementation and deployment views.
3. **Object Diagram:** Represents instances of classes and the relationships between them at a particular moment, providing a snapshot of the system state.
4. **Package Diagram:** Groups related elements into packages, showing their organization and dependencies, often used for organizing the system structure.
5. **Profile Diagram:** Allows for customization of UML models for specific domains by defining custom stereotypes, tagged values, and constraints.

Aspect	Behavioural Diagrams	Structural Diagrams
Purpose	Capture dynamic behaviour and interactions within the system.	Represent static aspects and organization of the system.
Focus	Processes, workflows, interactions, and state changes over time.	System elements and their relationships, structure, and hierarchy.
What it Models	How the system behaves and responds to events.	How the system is organized and built structurally.
Examples	Activity Diagram, Sequence Diagram, Use Case Diagram, State Machine Diagram	Class Diagram, Component Diagram, Object Diagram, Package Diagram
System View	Shows " what the system does " in terms of actions and interactions.	Shows " how the system is built " and its organizational structure.
Key Elements	Objects, actions, messages, states, transitions.	Classes, components, packages, relationships.
Temporal Focus	Models time-based changes and responses to specific events or actions.	Models static structure that generally remains consistent over time.
Use Cases	Useful for visualizing workflows, state transitions, and user-system interactions.	Useful for visualizing class hierarchies, component dependencies, etc.
Diagram Type	Dynamic — emphasizes system behaviour over time	Static — emphasizes system organization
Complementary Nature	Shows interaction flow during execution	Supports behavioural diagrams by detailing structural dependencies

Static Models

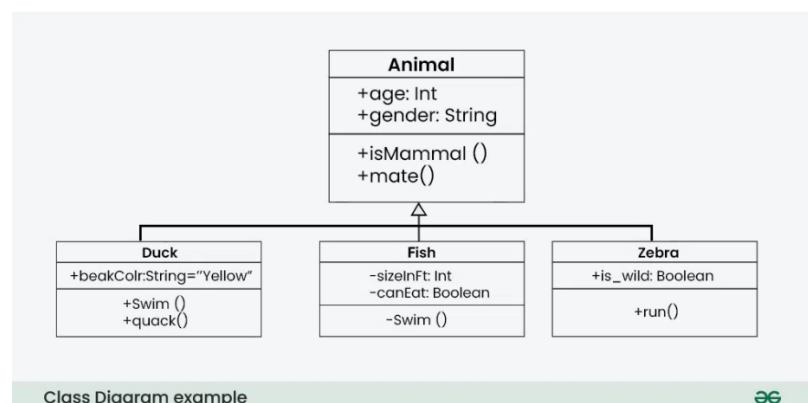
Static models in software engineering are used to represent the **structure** of a system. These models focus on aspects that remain unchanged during execution, such as components, relationships, data, and attributes. Static models provide a blueprint for the system's design, showing how the various parts of the system are organized and interact with each other at a high level. **Key Concepts:**

1. Purpose:

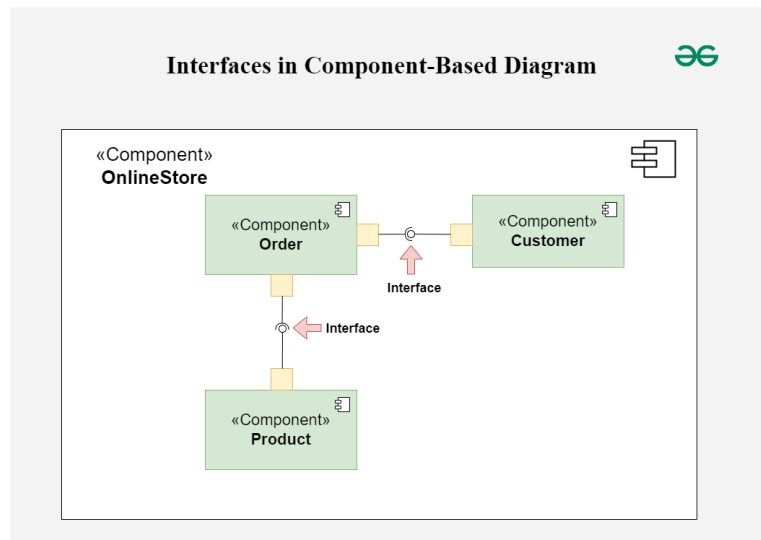
- To **define system components** and their relationships.
- To **structure the system**, ensuring that components are organized in a way that supports easy modification, maintenance, and scalability.
- To **document design decisions** for clarity and understanding across teams.

2. Types of Static Models:

- **Class Diagrams (Object-Oriented Modelling):** A class diagram is one of the most common types of static models in object-oriented systems. It shows the **classes** in the system, their **attributes**, and their **methods**, along with the relationships between classes (inheritance, association, etc.).



- **Component Diagrams:** These diagrams show how **components** in the system interact with each other. A component is a modular part of the system that can be independently replaced, tested, or maintained. The component diagram shows the **interfaces** and **dependencies** between components.



- **Data Flow Diagrams (DFD):** DFDs represent the flow of data within the system. They show how **data inputs** are processed to yield outputs. DFDs highlight the **processes**, **data stores**, and **external entities** that interact with the system.

3. Benefits of Static Models:

- Help to visualize the system's architecture early in the design process.
- Simplify the identification of system components and their relationships.
- Serve as a guide for developers during the coding phase.

Dynamic Models

Dynamic models focus on the **behaviour** of a system over time. These models are used to depict how the system responds to inputs, state changes, or external events during its execution. Dynamic models show how the system behaves & evolves representing the changing aspects of the system, such as **state transitions**, **method calls**, and **interactions** between system components. They describe how the system reacts to specific actions or inputs and how data or states change over time **Key Concepts:**

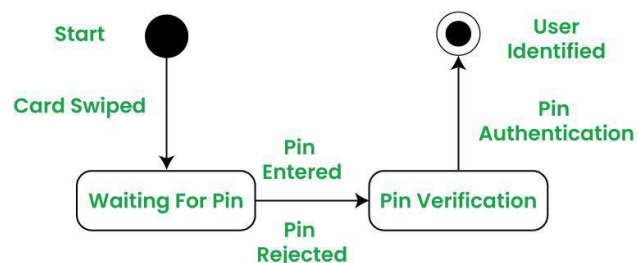
1. Purpose:

- To describe **system behaviour** in terms of inputs, outputs, and internal state changes.
- To simulate and understand the **dynamic interactions** within the system.
- To validate that the system's design will fulfil the requirements under various conditions.

2. Types of Dynamic Models:

- **State Diagrams (State Machine Diagrams):** State diagrams describe how an object or system **transitions** between different states in response to events. These diagrams show **states** and **state transitions**, helping to model the life cycle of an object or a system.

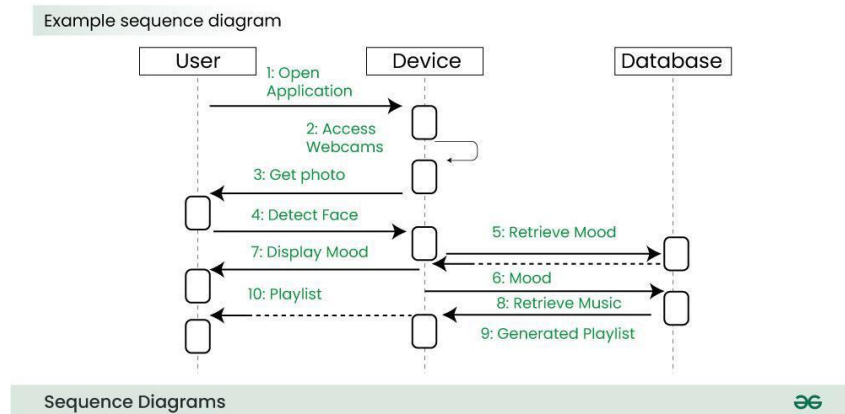
A State Machine Diagram for user verification



State Machine Diagrams | Unified Modeling Language (UML)

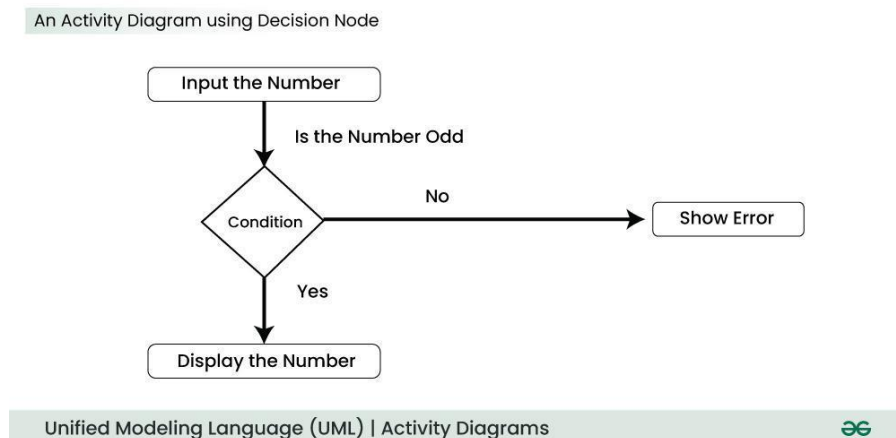
- **Sequence Diagrams:**

Sequence diagrams show the interaction between **objects** or **components** over time. They emphasize the order of messages or method calls and their sequencing in response to external events.



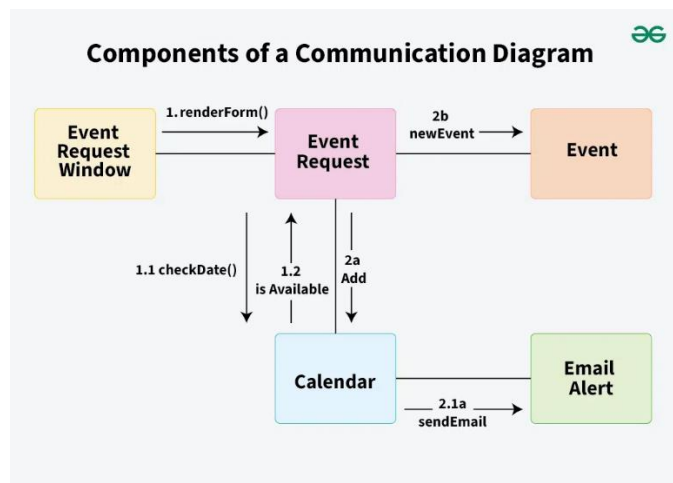
- **Activity Diagrams:**

Activity diagrams represent workflows or processes in the system. They show how various **activities** or **actions** are performed in sequence or in parallel, representing the flow of control and data.



- **Communication Diagrams (Collaboration Diagrams):**

These diagrams focus on the communication between different objects or components, showing how they interact and exchange messages during the execution of a system.



3. Benefits of Dynamic Models:

- Help in **testing** and **simulation** of system behaviour before actual implementation.
- Allow engineers to understand the system's **reaction** to various inputs and states.
- Facilitate the identification of potential problems in system behaviour, such as **race conditions**, **deadlocks**, or **bottlenecks**.

State and Activity Models

State and activity models are dynamic models used to represent **behaviour** over time. They help describe how the system changes its state in response to events or actions and how different processes are executed within the system. **Key Concepts:**

- **State Models:** Focus on the **states** that an object or system can occupy during its life cycle. They describe **transitions** between states triggered by specific **events** or **conditions**. State models help capture the dynamic aspects of a system, such as the transition of a software module between various modes or conditions.
- **Activity Models:** Show the flow of control and data within a system as a sequence of **activities** or **actions**. These models are concerned with how **tasks** or **functions** are performed, including conditions that trigger the flow of one activity to the next.

1. State Models or State Diagram (State Machine):

- Represents the various **states** an object or component can be in during its life cycle, and the **events** or **conditions** that cause transitions between states. These models show both the **current state** and **next state** in response to inputs.
- Example: A ticket booking system may have states such as "Available," "Booked," "Cancelled," etc., with transitions triggered by actions like a user booking a ticket or cancelling the booking.

2. Activity Models or Activity Diagram:

- Visualizes the workflow or sequence of actions in the system. It represents tasks or activities, the **flow of control**, and **decisions** made during the process. Activity diagrams can show both **sequential** and **parallel** activities.
- Example: In an e-commerce application, an activity diagram might illustrate the steps in the order processing workflow—such as "Add to Cart," "Checkout," "Payment," "Shipping," etc.

3. Benefits of State and Activity Models:

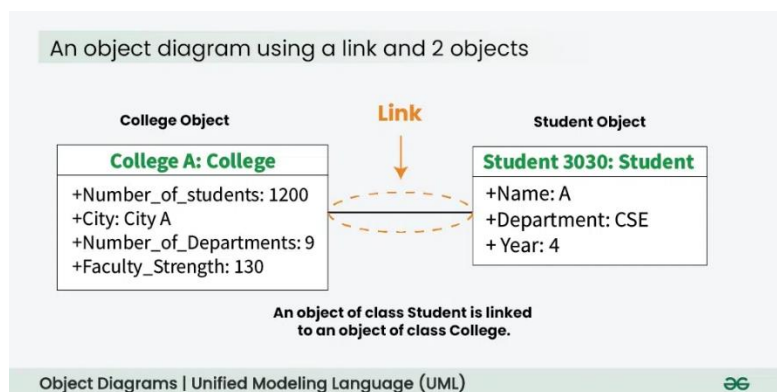
- Provide clarity on how a system behaves in different states or conditions.
- Help in **visualizing workflows** and decision points.
- Allow for the identification of bottlenecks or errors in the flow or state transitions.
- Useful in **test case design** and validating that the system behaves as expected.

Object-Oriented Modelling (Classes, Objects, and Messages)

Object-Oriented Modelling (OOM) is used to represent a system's structure and behaviour using the fundamental concepts of **objects** and **classes**. In OOM, the system is divided into **objects** that interact with each other to perform tasks and process information. **Key Concepts:**

• Classes and Objects:

- A **class** is a blueprint or template for creating objects. It defines the **attributes** (data) and **methods** (functions) that an object of that class will have. An **object** is an instance of a class, representing a specific entity in the system with actual values for its attributes.
- Example: A "Car" class may have attributes like **colour**, **make**, **model**, and **methods** like **drive ()** or **stop()**. An object of the "Car" class could be a specific car with a set colour, make, and model.



- **Messages (Method Calls):**
 - Objects interact with one another by sending **messages** (method calls). When an object needs to perform an action or retrieve information from another object, it sends a message, often invoking a method defined in the receiving object's class.
 - Example: A "Car" object may send a message to a "Driver" object to call the method **drive ()** when the car is ready to move.
- **Encapsulation:** The concept of **encapsulation** in OOM refers to hiding the internal details of an object (e.g., its state and behaviour) while exposing only the **necessary interface** (e.g., public methods) for interacting with the object.
- **Inheritance and Polymorphism:**
 - **Inheritance** allows one class to inherit the properties and behaviours of another class, enabling reusability and extension of functionality. **Polymorphism** allows objects of different classes to be treated as objects of a common superclass, enabling flexibility in handling different types of objects.

Benefits of Object-Oriented Modelling:

- Promotes **reusability** and **modularity** by defining components as objects.
- **Encapsulation** improves system security and reduces complexity by hiding internal implementation details.
- Supports **inheritance** and **polymorphism**, which make it easier to extend the system and accommodate new requirements or changes.
- Makes it easier to model real-world entities and their relationships, simplifying the design process.